Machine Learning: A Comprehensive Textbook

Solutions to Chapter 1 Exercises

Introduction to Machine Learning

Solutions Manual

Exercise 1.1 — Problem Formulation

A hospital wants to predict which patients will be readmitted within 30 days of discharge. Formally specify the task T, performance measure P, and experience E using Mitchell's definition. Discuss why accuracy is a poor choice of P here, and propose a better alternative.

Solution. Using Mitchell's definition (a program learns from experience E with respect to task T and performance measure P if its performance at T, measured by P, improves with E):

Task T: Given a feature vector x describing a patient at discharge (diagnoses, labs, medications, demographics, prior admissions, length of stay, etc.), predict $\hat{y} = f(x) \in \{0, 1\}$, where $y = 1$ indicates the patient will be readmitted within 30 days.

Experience E: A historical dataset $\{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$ of past discharges with observed 30-day readmission outcomes.

Performance measure P: A metric evaluated on held-out patients, chosen to reflect the clinical decision problem (see below).

Why accuracy is poor here. Readmission is a rare event (typically 10–20% base rate). A trivial classifier that always predicts "no readmission" can achieve 80–90% accuracy while being clinically useless — it never identifies a single at-risk patient. Accuracy treats false negatives (missed at-risk patients, who may suffer preventable harm) and false positives (unnecessary follow-up resources) as equally costly, which is not true in this setting: missing a readmission is typically far more costly than an unnecessary intervention.

Better alternatives:

- **Recall (sensitivity) at a fixed precision/budget**, or the F_β-score with $\beta > 1$ to weight recall more heavily, since failing to flag a true readmission is the costlier error.
- **AUROC / AUPRC**, which are threshold-independent and, in the imbalanced case, AUPRC is more informative than AUROC because it focuses on the positive (minority) class.
- **Expected cost**: Cost $= c_{FN}\cdot\text{FN} + c_{FP}\cdot\text{FP}$, with c_{FN}, c_{FP} elicited from clinicians/administrators, which directly optimizes for the decision at hand (see Exercise 1.6 for the general threshold formula).

Exercise 1.2 — Supervised vs Unsupervised

Classify each scenario as supervised, unsupervised, semi-supervised, self-supervised, or reinforcement learning, and justify.

(a) Training AlphaGo to play Go by self-play.

(b) Grouping 10M news articles by topic with no labels.

(c) Predicting tomorrow's electricity demand from historical consumption.

(d) Training BERT by masking 15% of tokens.

(e) Recommending movies from a user's previous ratings.

Solution.

(a) **Reinforcement learning.** There are no labeled "correct moves"; the agent takes actions (moves), receives a sparse terminal reward (win/loss, ± 1), and updates a policy to maximize expected cumulative reward through trial-and-error interaction with the (self-play) environment.

(b) **Unsupervised learning.** No topic labels exist; the goal is to discover latent structure (clusters/topics) directly from the feature distribution of the documents, e.g. via k-means or topic models.

(c) **Supervised learning (regression).** Historical consumption values act as labels: each day's demand $y^{(i)}$ is a known target to be predicted from features $x^{(i)}$ (past consumption, weather, calendar effects) via a regression model trained to minimize a loss like MSE.

(d) **Self-supervised learning.** The masked tokens serve as labels automatically generated from the unlabeled corpus itself (predict the identity of the masked token from context) — no external human annotation is required, but the training objective still has the supervised (predict-the-target) form.

(e) **Supervised learning**, specifically collaborative filtering framed as (typically) regression on observed ratings, since the ratings given by the user are explicit labeled targets used to predict ratings on unseen movies. (If one instead only had implicit signals like clicks with no explicit ratings, it would drift toward self-supervised/unsupervised framings, but as stated — "ratings are given" — the labels exist.)

Exercise 1.3 — No Free Lunch (NFL) Theorem

(a) What does the NFL theorem say about the claim "deep networks are always best"? (b) Give a concrete problem where a linear model provably outperforms a neural network.

Solution. **(a)** The NFL theorem states that, averaged uniformly over *all* possible target functions (data-generating distributions), every learning algorithm has the same expected generalization performance. Formally, if $\mathcal{A}_1, \mathcal{A}_2$ are two algorithms and we average the off-training-set error over all possible labelings/target functions f, then

$$\sum_f \mathrm{Err}(\mathcal{A}_1 \mid f) = \sum_f \mathrm{Err}(\mathcal{A}_2 \mid f).$$

No algorithm is universally superior; an algorithm's apparent superiority always reflects an implicit *inductive bias* that happens to match the structure of the problems we actually care about. So "deep networks are best" can only be true relative to the (non-uniform, structured) distribution of real-world tasks, not as a universal statement, and there necessarily exist problems for which simpler models are provably better matched.

(b) Consider a genomics problem with $n = 50$ patients and $d = 20{,}000$ gene-expression features, where the true relationship is $y = \theta^{*\top} x + \epsilon$ with θ^* sparse (only a handful of genes matter) and Gaussian noise. Here $n \ll d$:

- A linear model with ℓ_1 regularization (LASSO) has an inductive bias (sparsity) that exactly matches the true generative process, and enjoys well-understood finite-sample recovery guarantees under this regime.
- A neural network has vastly more parameters than samples; even with regularization/dropout, empirically and theoretically (VC-dimension / Rademacher complexity arguments, Section 1.6) it is far more prone to overfitting in the $n \ll d$ regime, and it discards the interpretability required for downstream clinical validation.

Hence for this data-generating process, the linear model provably attains lower expected test risk than an unconstrained deep network — precisely because its inductive bias matches the problem, illustrating NFL in practice.

Exercise 1.4 — Sample Complexity Derivation

Prove the PAC sample complexity bound $n \geq \frac{1}{\epsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right)$ from scratch, using only the union bound and Hoeffding's inequality.

Solution. **Setup.** Let $\mathcal{H}$ be a finite hypothesis class, and suppose the target concept $c^* \in \mathcal{H}$ is realizable (i.e. there is a hypothesis with zero true risk). For $h \in \mathcal{H}$ let $R(h) = \Pr_{(x,y)\sim\mathcal{D}}[h(x) \neq y]$ be the true risk and $\hat{R}(h) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[h(x^{(i)}) \neq y^{(i)}]$ the empirical risk on n i.i.d. samples. Let $h_S = \arg\min_{h\in\mathcal{H}} \hat{R}(h)$ be the ERM hypothesis. We want: with probability $\geq 1-\delta$, $R(h_S) \leq \epsilon$.

Step 1: Bad hypotheses. Call h "ϵ-bad" if $R(h) > \epsilon$. Let $\mathcal{H}_{\text{bad}} = \{h \in \mathcal{H} : R(h) > \epsilon\}$. We show that with high probability, no ϵ-bad hypothesis has zero empirical error (hence ERM, which only selects hypotheses with the smallest empirical error, cannot output a bad one — since the realizable target c^* achieves $\hat{R}(c^*) = 0$, ERM's chosen h_S also satisfies $\hat{R}(h_S) = 0$).

Step 2: Hoeffding's inequality for a single bad hypothesis. For a fixed $h \in \mathcal{H}_{\text{bad}}$, $n\hat{R}(h) = \sum_i \mathbf{1}[h(x^{(i)}) \neq y^{(i)}]$ is a sum of n i.i.d. Bernoulli($R(h)$) variables. By Hoeffding's inequality,

$\Pr[\hat{R}(h) = 0] \leq \Pr[\hat{R}(h) \leq R(h) - \epsilon']$ is not directly what we need; instead we bound directly:

$$\Pr[\hat{R}(h) = 0] = (1 - R(h))^n \leq (1-\epsilon)^n \leq e^{-\epsilon n},$$

using $R(h) > \epsilon$ and the elementary inequality $1 - x \leq e^{-x}$.

Step 3: Union bound over $\mathcal{H}$. We want the probability that *some* bad hypothesis has zero empirical error:

$$\Pr\left[\exists h \in \mathcal{H}_{\text{bad}} : \hat{R}(h) = 0\right] \leq \sum_{h\in\mathcal{H}_{\text{bad}}} \Pr[\hat{R}(h) = 0] \leq |\mathcal{H}|\, e^{-\epsilon n}.$$

Step 4: Solve for n. We require this failure probability to be at most δ:

$$|\mathcal{H}|e^{-\epsilon n} \leq \delta \iff e^{-\epsilon n} \leq \frac{\delta}{|\mathcal{H}|} \iff -\epsilon n \leq \ln\delta - \ln|\mathcal{H}| \iff \epsilon n \geq \ln|\mathcal{H}| - \ln\delta = \ln|\mathcal{H}| + \ln\frac{1}{\delta}.$$

Hence

$$n \geq \frac{1}{\epsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right)$$

suffices to guarantee that, with probability at least $1-\delta$ over the draw of the sample, every hypothesis with zero empirical error also has true risk $\leq \epsilon$ — in particular this holds for the ERM hypothesis h_S. ■

Exercise 1.5 — VC Dimension

(a) Prove VCdim of linear classifiers in $\mathbb{R}^d$ is exactly $d+1$. (b) What is the VC dimension of k-NN, and what does it imply about bias-variance?

Solution. **(a) Shattering $d+1$ points.** Take the $d+1$ points $x_0 = 0, x_1 = e_1, \ldots, x_d = e_d$ (the origin and the standard basis vectors), or more simply any set of $d+1$ points in *general position* (affinely independent). A linear classifier is $h_{w,b}(x) = \text{sign}(w^\top x + b)$. Given any labeling $y_0, \ldots, y_d \in \{-1, +1\}$ of $d+1$ affinely independent points, one can always find (w, b) achieving it: augment each x_i with a constant 1 to get $\tilde{x}_i = (x_i, 1) \in \mathbb{R}^{d+1}$; affine independence of the x_i is equivalent to linear independence of the $\tilde{x}_i$ in $\mathbb{R}^{d+1}$. Since the $\tilde{x}_i$ are $d+1$ linearly independent vectors in $\mathbb{R}^{d+1}$, they form a basis, so the linear map $\theta \mapsto (\tilde{x}_0^\top\theta, \ldots, \tilde{x}_d^\top\theta)$ (with $\theta = (w, b)$) is a bijection $\mathbb{R}^{d+1} \to \mathbb{R}^{d+1}$; hence we can choose θ so that $\tilde{x}_i^\top\theta = y_i$ exactly (any target vector, in particular any sign pattern, is attainable), which certainly realizes $\text{sign}(\tilde{x}_i^\top\theta) = y_i$. Thus all 2^{d+1} labelings are realizable: $d+1$ points can be shattered.

No $d+2$ points can be shattered. Let $x_1, \ldots, x_{d+2}$ be any $d+2$ points in $\mathbb{R}^d$. By **Radon's theorem**, any $d+2$ points in $\mathbb{R}^d$ can be partitioned into two disjoint sets I, J (a "Radon partition") whose convex hulls intersect: $\text{conv}(\{x_i\}_{i\in I}) \cap \text{conv}(\{x_j\}_{j\in J}) \neq \varnothing$. Consider the labeling that assigns $y_i = +1$ for $i \in I$ and $y_j = -1$ for $j \in J$. Suppose for contradiction a linear classifier $h_{w,b}$ realizes this labeling, i.e. $w^\top x_i + b > 0$ for $i \in I$ and $w^\top x_j + b < 0$ for $j \in J$ (strict separation is required to realize a labeling with the sign function, and the boundary case is handled by an infinitesimal perturbation argument). Since $\{x : w^\top x + b > 0\}$ is convex, $\text{conv}(\{x_i\}_{i\in I}) \subseteq \{w^\top x + b > 0\}$; likewise $\text{conv}(\{x_j\}_{j\in J}) \subseteq \{w^\top x + b < 0\}$. But these two convex hulls share a common point p by Radon's theorem, and p cannot simultaneously satisfy $w^\top p + b > 0$ and $w^\top p + b < 0$ — contradiction. Hence this particular labeling cannot be realized by any linear classifier, so no set of $d+2$ points can be shattered.

Combining both directions: $\text{VCdim}(\text{linear classifiers in } \mathbb{R}^d) = d+1$.

(b) VC dimension of k-NN. For $k = 1$ (1-NN), the VC dimension is *infinite*: given any n points in general position with any labeling, the 1-NN rule classifies each training point as itself (distance 0), so it perfectly memorizes (shatters) an arbitrarily large set of points. More generally, for fixed small k, k-NN can also shatter arbitrarily large point sets (each region around a point is dominated by its own label for suitably separated points), so $\text{VCdim}(k\text{-NN}) = \infty$ for any fixed k independent of n.

Implication. An infinite VC dimension means k-NN (especially with small k) is an extremely low-bias, high-variance, high-capacity model class: it can perfectly fit (interpolate) essentially any finite training set, so it is prone to overfitting, and generalization must instead be controlled not by hypothesis-class complexity bounds but via k (which effectively acts as capacity control — larger k smooths the decision boundary, trading variance for bias) and via consistency results that hold as $n \to \infty$, $k \to \infty$, $k/n \to 0$.

Exercise 1.6 — The Metric Trap

A fraud system processes 10M transactions/day at 0.01% positive rate. A "never fraud" model achieves 99.99% accuracy. (a) Compute its precision, recall, F1. (b) Optimal decision threshold given costs \$500 (FN) and \$10 (FP). (c) Design an asymmetric-cost loss.

Solution. **(a)** The "always predict not-fraud" model never predicts the positive class, so $TP = 0$ and FN = all actual frauds. Precision $= TP/(TP + FP) = 0/0$ is **undefined** (conventionally

reported as 0 by scikit-learn with a warning, since there are no positive predictions to be precise about). Recall $= TP/(TP+FN) = 0/(\text{all frauds}) = \mathbf{0}$. $F_1 = 2 \cdot \frac{P \cdot R}{P+R}$ is likewise **0** (or undefined) — despite 99.99% accuracy, the model catches *zero* fraud, which is the concrete illustration of why accuracy is a trap under extreme imbalance.

(b) Let $\pi =$ prior fraud rate $= 10^{-4}$, and let a classifier output score $p(x) = \Pr[\text{fraud} \mid x]$, predicting fraud when $p(x) > \tau$. The expected cost of flagging at threshold τ for a case with true score p is minimized pointwise by comparing the cost of a false negative vs. false positive: predict fraud iff

$c_1 \cdot (1-p) \cdot$[cost if actually fraud and we say "not fraud"] $\lessgtr$ $c_0 \cdot p \cdot$[cost if actually not fraud and we say "fraud"]

More directly, using the standard Bayes-risk threshold derivation (see also Exercise 7.3): predict positive iff the expected cost of predicting positive is lower than predicting negative,

$$c_0(1-p) < c_1\, p \iff p > \frac{c_0}{c_0 + c_1}.$$

This gives a threshold on the *posterior* $p(x)$, not the raw prior. If the model is well-calibrated with prior π, the threshold in terms of a score assumed proportional to the base rate is

$$\tau^* = \frac{c_0}{c_0 + c_1} = \frac{10}{10 + 500} = \frac{10}{510} \approx 0.0196.$$

Interpretation: because a missed fraud ($c_1 = \$500$) is 50× costlier than a false alarm ($c_0 = \$10$), the optimal threshold on the predicted *posterior probability of fraud* is far below 0.5 — flag a transaction as fraud whenever the model believes there's even a $\sim 2\%$ chance it's fraudulent. Note this threshold does not depend directly on the prior π if $p(x)$ is a properly calibrated posterior (the prior is already "baked into" $p(x)$ via Bayes' rule); if instead one is thresholding a likelihood ratio or an uncalibrated score, the prior enters explicitly and τ^* shifts proportionally to π (lower prior $\to$ lower absolute count of frauds $\to$ same relative threshold on the calibrated posterior, but a much more conservative threshold if working with raw likelihood ratios).

(c) A custom asymmetric loss (cost-sensitive cross-entropy):

$$\mathcal{L}(y, \hat{p}) = -\big[c_1\, y \log \hat{p} + c_0\, (1-y) \log(1-\hat{p})\big],$$

with $c_1 = 500, c_0 = 10$ (or normalized, $c_1/c_0 = 50$), so that missing a fraud is penalized 50× more heavily during training than raising a false alarm — this directly shapes the decision boundary/gradient towards higher recall on the rare class, complementing threshold-moving at inference time.

Exercise 1.7 — Data Leakage

A data scientist standardises using statistics from the *entire* dataset before splitting. (a) Is this leakage — what leaks? (b) Give a numerical example of overestimated performance. (c) Correct procedure.

Solution. **(a)** Yes, this is data leakage. The standardization statistics $\hat{\mu}, \hat{\sigma}$ are computed as $\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} x^{(i)}$, $\hat{\sigma}^2 = \frac{1}{N}\sum_{i=1}^{N}(x^{(i)} - \hat{\mu})^2$ over *all* N points, including the test set. Every test point therefore influences the mean/variance used to transform *every* training point (and itself). This means information about the test set's distribution (its actual values) has leaked into the preprocessing pipeline that produces the training features — violating the assumption that the test

set is unseen during model development. In effect the model's inputs are "aware" of statistics that a truly deployed system, seeing only past training data, would not have access to.

(b) Concrete example: suppose the true population is $x \sim \mathcal{N}(0,1)$, and by chance a small training split of $n_{tr} = 20$ points has empirical mean $\bar{x}_{tr} = 0.5$ (a plausible fluctuation), while $n_{te} = 20$ test points have $\bar{x}_{te} = -0.5$. If standardization statistics are computed correctly on the training set only, test points are centered using $\bar{x}_{tr} = 0.5$, so the test-set feature distribution appears *shifted* relative to training — a train/test mismatch that a model must be robust to, exactly mirroring what will happen in deployment (train stats fixed, new unseen data with its own random offset). But if standardization instead uses the full-dataset mean $\bar{x}_{all} = \frac{20(0.5)+20(-0.5)}{40} = 0$, the observed train/test shift is artificially erased — the pipeline has "peeked" at the test set to cancel out precisely the kind of distributional discrepancy the held-out evaluation is meant to detect. Numerically, in a downstream model sensitive to feature shift (e.g. a linear model with a fixed decision boundary calibrated on the correctly-shifted training features), the leaked version can show test accuracy several points higher than an honest correctly-split pipeline, because the leakage effectively lets the test set correct its own distribution before evaluation — the reported metric no longer reflects performance on genuinely unseen data.

(c) Correct procedure: split first, then fit the scaler (compute $\hat{\mu}, \hat{\sigma}$) using *only* the training fold, and apply that fixed transform to both the training and test folds (and, in cross-validation, refit the scaler independently within each fold on that fold's training portion). More generally: any statistic used for preprocessing (imputation values, encoding maps, PCA components, feature-selection decisions) must be fit exclusively on the training partition inside each CV fold, and applied (not refit) to the corresponding validation/test partition.

Exercise 1.8 — Research Question: VC Bound for GPT-3

The VC dimension of ReLU networks is $O(WL \log W)$. For GPT-3 ($W = 175$B, $L = 96$), compute the VC bound on the generalization gap. What does this say about classical learning theory for LLMs?

Solution. The classical generalization bound (Section 1.6) states that with probability $\geq 1 - \delta$,

$$R(h) \leq \hat{R}(h) + O\left(\sqrt{\frac{\text{VCdim}(\mathcal{H}) \log n + \log(1/\delta)}{n}}\right).$$

With $\text{VCdim} = O(WL \log W)$, $W = 1.75 \times 10^{11}$, $L = 96$:

$$\text{VCdim} \approx WL \log W = (1.75 \times 10^{11})(96) \log(1.75 \times 10^{11}).$$

Using $\log(1.75 \times 10^{11}) \approx \ln(1.75 \times 10^{11}) \approx 25.9$ (natural log; the constant inside the bound is typically base-agnostic up to constants):

$$\text{VCdim} \approx (1.75 \times 10^{11})(96)(25.9) \approx 4.35 \times 10^{14}.$$

GPT-3 was trained on roughly $n \approx 3 \times 10^{11}$ tokens (a few hundred billion). Plugging into the bound (treating each token/context as one "example" for a rough order-of-magnitude estimate):

$$\sqrt{\frac{\text{VCdim}}{n}} \approx \sqrt{\frac{4.35 \times 10^{14}}{3 \times 10^{11}}} = \sqrt{1450} \approx 38.$$

This bound is **vacuous**: it predicts a generalization gap of order 38 (in whatever units the loss is measured, e.g. many times larger than the loss itself, which is $O(1)$ nats/token), i.e. it gives no meaningful guarantee whatsoever — the bound is many orders of magnitude larger than the actual observed train/test gap for GPT-3, which is empirically small.

Interpretation. This is a well-known and important disconnect: classical VC-dimension-based uniform-convergence bounds count the *raw parametric capacity* of the hypothesis class and are agnostic to (a) the optimization trajectory (SGD/Adam only ever visits a tiny, implicitly-regularized subset of the full parameter space reachable in principle), (b) the structure of natural language data (which has far more regularity than an adversarial worst-case sequence), and (c) phenomena like double descent, benign overfitting, and implicit regularization from architecture and optimizer choices, none of which classical VC theory accounts for. This is precisely why deep learning theory has moved toward alternative frameworks — margin-based bounds, PAC-Bayes bounds, norm-based (not parameter-count-based) complexity measures, compression bounds, and the — still largely open — question of why heavily overparameterized networks generalize at all. The exercise is explicitly flagged as an open research question because there is, as of this writing, no tight, non-vacuous a priori generalization bound that matches observed LLM behavior.